

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



Proyecto de Primera Unidad

Vertex Cover GRUPO 01

Asignatura: Algoritmos Avanzados
Docente: Héctor Eduardo Ugarte Rojas
Semestre: 2026-II

Integrantes:
Caballero Zegarra, Steffano
Llactahuamani Ayala, Adriel
Morocco Amanca, Albert
Quispe Castilla, Diego

Cusco – Perú

2026

Optimización de la Ubicación de Sistemas de Detección de Intrusos

mediante Cobertura de Vértices

Comparación experimental entre una heurística greedy por grado y una 2-aproximación por emparejamiento maximal

October 7, 2026

Resumen

Este informe documenta el estado del proyecto grupal sobre **Cobertura de Vértices (Vertex Cover)** al cierre de la Primera Entrega, integrando los tres avances presentados durante el periodo (formulación del problema, diseño algorítmico con plan de validación, y prototipo mínimo ejecutable). El problema se aplica al contexto de ciberseguridad y monitoreo de redes: ubicar sistemas de detección de intrusos (IDS) en el número mínimo de routers tal que todo enlace de la red quede monitoreado por al menos uno de sus extremos. Se implementaron y compararon tres estrategias algorítmicas – una heurística greedy por grado, una 2-aproximación por emparejamiento maximal con garantía matemática demostrable, y una solución exacta de referencia por fuerza bruta – sobre un sistema reproducible, versionado en Git, con 27 pruebas automatizadas (casos básicos, límite y adversarios) y un primer experimento comparativo. El hallazgo más relevante del periodo fue un defecto de no-determinismo en la implementación inicial (dependiente del orden de iteración de los conjuntos de Python), detectado mediante las propias pruebas y corregido antes del cierre de la entrega, documentado aquí como evidencia de aprendizaje autónomo y pensamiento crítico. Los resultados preliminares muestran que el greedy iguala al óptimo en la mayoría de las instancias de prueba, mientras que el matching maximal se mantiene siempre dentro de su cota teórica de $2\times$ el óptimo; el algoritmo exacto confirma un crecimiento exponencial del tiempo de ejecución ya visible en instancias de menos de 20 vértices. El informe cierra señalando explícitamente lo pendiente para la Entrega 2: un solver exacto escalable por programación lineal entera, una metodología experimental ampliada y la optimización de la complejidad del algoritmo greedy.

Índice

1	Resumen	5
2	Introducción	6
2.1	Contexto y motivación	6
2.2	El problema	6
3	Objetivos	7
4	Alcance de esta entrega	8
5	Organización del documento	9
6	Fundamentos y aprendizaje autónomo	10
6.1	Conceptos y algoritmos requeridos	10
6.2	Fuentes estudiadas y criterios de selección	10
6.3	Conocimientos nuevos que el grupo tuvo que adquirir	10
6.4	Estrategia y evidencias de aprendizaje autónomo	11
7	Evaluación y selección del stack tecnológico	11
7.1	Alternativas consideradas y criterios de comparación	11
7.2	Procedimiento de aprendizaje e integración	12
7.3	Dificultades de adopción y soluciones aplicadas	13
8	Formulación del problema	14
8.1	Modelo	14
8.2	Entradas	14
8.3	Salidas	14
8.4	Restricciones	14
8.5	Supuestos	14
8.6	Función objetivo	14
8.7	Ejemplo: instancia pequeña resuelta manualmente	14
9	Diseño algorítmico	16
9.1	Greedy por grado (<code>greedy_degree_vertex_cover</code>)	16
9.2	Matching maximal / 2-aproximación (<code>maximal_matching_vertex_cover</code>)	16
9.3	Exacto (fuerza bruta de referencia) (<code>exact_vertex_cover_bruteforce</code>)	17
10	Diseño e implementación del sistema	18
10.1	Arquitectura y módulos	18
10.2	Flujo de información	18
10.3	Estructura de datos	18
10.4	Formato de archivo de instancia	18
10.5	Dependencias y decisiones técnicas	19
11	Estrategia de pruebas	20
11.1	Criterio de verificación de salidas	20
11.2	Pruebas de integración y validación cruzada	20
11.3	Resultado de la ejecución	20
12	Metodología experimental	21
12.1	Preguntas de evaluación	21
12.2	Hipótesis de trabajo	21

12.3	Instancias, variables y métricas	21
12.4	Entorno y procedimiento reproducible	21
13	Resultados	22
13.1	Validación contra la instancia manual	22
13.2	Experimento comparativo	22
14	Discusión crítica	23
14.1	Interpretación de los resultados frente a la teoría	23
14.2	El hallazgo del periodo: no determinismo en la implementación inicial	23
14.3	Limitaciones y amenazas a la validez	23
14.4	Cuándo conviene cada alternativa	24
15	Conclusiones y recomendaciones	25
15.1	Respuesta a los objetivos	25
15.2	Síntesis de hallazgos	25
15.3	Lecciones aprendidas	25
15.4	Trabajo futuro (Entrega 2)	25
16	Referencias	26
A	Anexos	27
A.1	Manual de instalación	27
A.2	Manual de uso	27
A.3	Bitácora de aprendizaje (resumen)	27
A.4	Evidencia adicional	27

1 Resumen

Ver el Resumen en la primera página. Esta sección numerada se incluye para respetar el índice de contenidos exigido por la guía del proyecto (*Contenidos del informe final*, ítem 1); su contenido es el mismo abstract presentado arriba, ya que en esta primera entrega no hay una sección separada que condense el documento con un alcance distinto.

2 Introducción

2.1 Contexto y motivación

En una red de comunicaciones, cada enlace físico entre dispositivos (routers, switches) es un punto potencial de tráfico malicioso o de fallas. Instalar un sistema de detección de intrusos (IDS) o firewall en *todos* los routers de la red garantiza visibilidad completa del tráfico, pero resulta costoso e innecesario en equipos, licencias y mantenimiento. Basta con que cada enlace tenga *al menos uno* de sus dos extremos monitoreado para que el tráfico que circula por él pueda inspeccionarse. Este planteamiento corresponde exactamente al problema clásico de **Cobertura de Vértices (Vertex Cover)**: dado un grafo no dirigido, encontrar el subconjunto mínimo de vértices tal que toda arista tenga al menos un extremo en dicho subconjunto.

2.2 El problema

Vertex Cover es NP-difícil en su formulación general [1], por lo que no se conoce (ni se espera encontrar, bajo la conjetura $P \neq NP$) un algoritmo que resuelva todas las instancias de forma óptima en tiempo polinomial. Esto hace que el problema sea un caso de estudio natural para **comparar estrategias**: algoritmos exactos (correctos pero costosos), heurísticos (rápidos pero sin garantía) y de aproximación con cota demostrable (rápidos y con garantía parcial). Esa comparación – no la implementación de un único algoritmo – es el eje central exigido por la línea de trabajo asignada al grupo ("cobertura de vértices y comparación de estrategias de aproximación").

3 Objetivos

Objetivo general

Desarrollar y evaluar experimentalmente una solución computacional para el problema de Cobertura de Vértices aplicado a la ubicación de sistemas de monitoreo en redes, demostrando aprendizaje autónomo, adaptación a herramientas pertinentes y pensamiento crítico en la selección, implementación y análisis de alternativas algorítmicas.

Objetivos específicos

1. Formular el problema mediante entradas, salidas, restricciones, supuestos y función objetivo claramente definidos (Avance 1).
2. Diseñar la arquitectura del sistema (módulos, estructuras de datos, formatos de archivo) y documentar el pseudocódigo y análisis de complejidad de cada algoritmo (Avance 2).
3. Implementar al menos dos algoritmos comparables – se implementaron tres – y un mecanismo de referencia exacto para validar instancias pequeñas.
4. Diseñar y ejecutar un plan de pruebas que cubra casos básicos, límite y adversarios, con un criterio de verificación explícito de las salidas.
5. Construir un prototipo mínimo ejecutable, demostrable en vivo con una instancia conocida o una propuesta en el momento (Avance 3).
6. Diseñar experimentos reproducibles que comparen tiempo de ejecución y calidad de solución (ratio de aproximación empírico) entre los algoritmos.
7. Evaluar críticamente las herramientas candidatas, los resultados obtenidos, los supuestos y las limitaciones del sistema actual.
8. Documentar el proceso de aprendizaje autónomo del grupo a lo largo de las tres entregas progresivas.

4 Alcance de esta entrega

Este informe cubre lo exigido para la **Primera Entrega**: formulación del problema, diseño del sistema con su plan de validación, y un prototipo mínimo ejecutable con resultados experimentales *preliminares*. No incluye todavía: el solver exacto escalable por programación lineal entera (ILP), la metodología experimental ampliada a múltiples densidades y topologías, ni la defensa individual, que corresponden a la Entrega 2 y a la Entrega 3 respectivamente. Esto se señala explícitamente en cada sección donde aplica, en vez de presentar resultados parciales como si fueran definitivos.

5 Organización del documento

La Sección 6 presenta los fundamentos teóricos y el proceso de aprendizaje autónomo del grupo. La Sección 7 evalúa y justifica el stack tecnológico. La Sección 8 formula el problema formalmente. Las Secciones 9 y 10 cubren el diseño algorítmico y el diseño del sistema. La Sección 11 describe la estrategia de pruebas. Las Secciones 12 y 13 presentan la metodología experimental y sus resultados. La Sección 14 discute críticamente esos resultados. La Sección 15 cierra con conclusiones y recomendaciones, seguida de referencias y anexos.

6 Fundamentos y aprendizaje autónomo

6.1 Conceptos y algoritmos requeridos

El desarrollo del proyecto requirió manejar con solvencia los siguientes conceptos:

- **Teoría de grafos básica:** grafos no dirigidos, grado de un vértice, cobertura de vértices, y su relación de dualidad con *Independent Set* ($V \setminus C$ es un conjunto independiente si y solo si C es una cobertura de vértices).
- **NP-completitud:** por qué Vertex Cover es NP-difícil [1], y qué implica eso para la estrategia de solución (no buscar un algoritmo exacto polinomial, sino comparar heurísticas, aproximaciones y un exacto acotado en tamaño).
- **Algoritmos de aproximación:** qué es una garantía de aproximación de factor k , y cómo se demuestra formalmente (ver la demostración de la cota $2\times$ del matching maximal en la Sección 9) [2].
- **Análisis de complejidad algorítmica:** notación $O(\cdot)$ para caracterizar el comportamiento de cada algoritmo en función de $|V|$ y $|E|$ [3].

6.2 Fuentes estudiadas y criterios de selección

Se priorizaron fuentes académicas y documentación oficial de las herramientas, siguiendo el criterio explícito de la guía del proyecto (instrucción adicional 11: "se priorizarán artículos, libros, documentación oficial y recursos técnicos verificables"):

- Material de referencia sobre NP-completitud y algoritmos de aproximación (Cormen et al., *Introduction to Algorithms* [3]; Vazirani, *Approximation Algorithms* [2]), elegidos por ser textos estándar y ampliamente citados en la literatura de algoritmos.
- Documentación oficial de `pytest`, `PuLP` y `NetworkX`, priorizada sobre tutoriales de terceros por ser la fuente autoritativa y vigente de cada herramienta.

6.3 Conocimientos nuevos que el grupo tuvo que adquirir

- La demostración formal de por qué el emparejamiento (*matching*) maximal produce una cobertura de a lo sumo el doble del tamaño óptimo – no bastaba con saber que el algoritmo existe, sino poder argumentar la cota.
- El comportamiento de la aleatorización de hash en Python (`PYTHONHASHSEED`) y su efecto sobre el orden de iteración de un `set()`, que resultó ser la causa de un defecto de reproducibilidad real en la implementación (detallado en la Sección 14).
- Formulación de programación lineal entera (ILP) para Vertex Cover, como base para incorporar un solver escalable en la Entrega 2 (minimizar $\sum_v x_v$ sujeto a $x_u + x_v \geq 1$ para toda arista (u, v) , con $x_v \in \{0, 1\}$).
- Uso de `pytest` con parametrización (`@pytest.mark.parametrize`) para repetir una misma verificación sobre múltiples instancias generadas aleatoriamente sin duplicar código de prueba.

6.4 Estrategia y evidencias de aprendizaje autónomo

El grupo mantiene una **bitácora de aprendizaje** (`docs/bitacora.md` en el repositorio) con una entrada por sesión real de trabajo, registrando qué se hizo, qué se tuvo que aprender o decidir por cuenta propia, y qué quedó pendiente. Esta bitácora es la evidencia principal del indicador AG-C06.01 y respalda, con fecha y contexto, los hallazgos que se resumen en este informe (en particular, el defecto de reproducibilidad de la Sección 14, que se documentó en el momento en que se encontró, no de forma retrospectiva). El Anexo A.3 reproduce un resumen de esa bitácora.

7 Evaluación y selección del stack tecnológico

7.1 Alternativas consideradas y criterios de comparación

Se aplicaron cinco criterios generales a toda decisión tecnológica: (1) facilidad de instalación sin dependencias del sistema operativo, (2) calidad y vigencia de la documentación oficial, (3) curva de aprendizaje razonable para el tiempo disponible, (4) adopción y soporte de la comunidad, y (5) que no añadiera complejidad innecesaria al núcleo algorítmico (instrucción adicional 5 de la guía: "la interfaz es un medio de interacción y no reemplaza el núcleo algorítmico").

Necesidad	Candidatas	Tecnología seleccionada y justificación
Lenguaje principal	Python 3 vs. Java vs. C++	Python 3: sintaxis concisa para prototipado algorítmico, <code>set</code> y <code>dict</code> nativos adecuados para listas de adyacencia, y es la configuración de referencia sugerida por la guía del curso.
Pruebas unitarias	<code>pytest</code> vs. <code>unittest</code> (estándar)	<code>pytest</code> : sintaxis de <code>assert</code> plano (más legible), soporte nativo de parametrización, amplia adopción y documentación.
Manejo y visualización de grafos	<code>NetworkX</code> vs. implementación propia pura	Implementación propia para el núcleo (control total sobre la complejidad, requisito pedagógico del curso); <code>NetworkX</code> queda como candidata para visualización en la Entrega 2.
Solver exacto escalable	PuLP vs. OR-Tools	PuLP (tecnología emergente del proyecto, pendiente de completar en la Entrega 2): instala el solver CBC junto con el paquete, sin dependencias del sistema; API declarativa simple para la formulación ILP estándar de Vertex Cover. OR-Tools se descarta por ahora por una curva de aprendizaje mayor frente al beneficio adicional en instancias de este tamaño.
Datos y gráficos	<code>pandas</code> + <code>matplotlib</code>	Estándar de facto, documentación extensa, suficiente para el volumen de datos de este proyecto.
Control de versiones	Git + GitHub	Exigido por la guía del proyecto (instrucción adicional 3); historial de commits como evidencia de evolución real del trabajo.
Documentación	$\text{\LaTeX}$ + Markdown	$\text{\LaTeX}$ para los documentos formales de cada entrega (control tipográfico de fórmulas, pseudocódigo y tablas); Markdown para la bitácora y notas operativas, por ser más rápido de editar y de revisar en diffs de Git.

7.2 Procedimiento de aprendizaje e integración

El flujo seguido para cada herramienta nueva fue: (1) leer la documentación oficial de inicio rápido, (2) escribir un script mínimo de prueba fuera del proyecto para confirmar el comportamiento esperado, (3) integrarla al proyecto real, y (4) cubrirla con al menos una prueba automatizada. Este procedimiento se siguió explícitamente para `pytest` (su sintaxis de parametrización se validó primero en un archivo de prueba aislado antes de usarse en `tests/test_solver.py`).

7.3 Dificultades de adopción y soluciones aplicadas

La dificultad de adopción más significativa del periodo no fue con una biblioteca externa, sino con el propio lenguaje: el comportamiento no determinístico de la iteración sobre `set()` en Python (detallado como hallazgo central en la Sección 14). Una segunda dificultad, menor, fue la imposibilidad de instalar el paquete de idioma español de L^AT_EX (`babel-spanish`) en el entorno de compilación usado para preparar los documentos de las tres entregas, sin acceso a instalarlo; se resolvió prescindiendo de `babel` y ajustando a mano los puntos donde la falta de guionado en español causaba desbordes de texto en tablas, priorizando – de nuevo según la instrucción adicional 5 – no bloquear la entrega por una herramienta no esencial al núcleo del proyecto.

8 Formulación del problema

8.1 Modelo

La topología de la red se representa como un grafo no dirigido $G = (V, E)$, donde:

- V es el conjunto de routers/switches de la red.
- E es el conjunto de enlaces físicos entre dichos dispositivos.

8.2 Entradas

- Un grafo $G = (V, E)$ que representa la topología de la red, en formato JSON (ver Sección 10).
- (Extensión futura, no implementada en esta entrega) costo de instalación por nodo, para una variante ponderada del problema.

8.3 Salidas

- Un subconjunto $C \subseteq V$: los routers donde se instalará el sistema de monitoreo.
- El tamaño $|C|$ obtenido por cada algoritmo, para fines comparativos.
- Verificación booleana de que C es una cobertura válida de E .

8.4 Restricciones

- Para toda arista $(u, v) \in E$ debe cumplirse $u \in C \vee v \in C$.
- $C \subseteq V$.

8.5 Supuestos

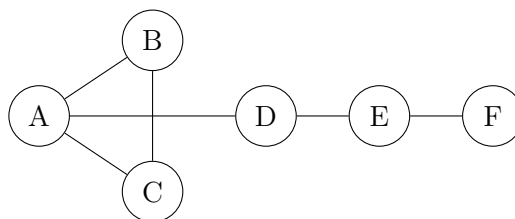
- La topología de la red es estática durante el cálculo de la solución.
- El grafo es no dirigido y no ponderado en esta primera fase del proyecto.
- Un router monitoreado detecta el tráfico de *todos* sus enlaces directos.

8.6 Función objetivo

$$\min_{C \subseteq V} |C| \quad \text{sujeto a} \quad \forall (u, v) \in E : u \in C \vee v \in C \quad (1)$$

8.7 Ejemplo: instancia pequeña resuelta manualmente

Se usa una mini-red de 6 routers como ejemplo ilustrativo, resuelta primero a mano (Avance 1) y luego reproducida con el sistema real (Sección 13):



$$V = \{A, B, C, D, E, F\}, \quad E = \{(A, B), (A, C), (A, D), (B, C), (D, E), (E, F)\}$$

Grados iniciales: $\deg(A) = 3$, $\deg(B) = 2$, $\deg(C) = 2$, $\deg(D) = 2$, $\deg(E) = 2$, $\deg(F) = 1$.

Greedy por grado: se elige A (grado 3), luego E (grado 2 entre las aristas restantes), luego B (o C) para cubrir la arista BC que queda pendiente. $C_{\text{greedy}} = \{A, E, B\}$, $|C_{\text{greedy}}| = 3$.

Matching maximal: se toma arbitrariamente (A, B) , agregando ambos extremos y eliminando todas las aristas que tocan a A o B ; de lo que queda se toma (D, E) , agregando ambos extremos. $C_{\text{match}} = \{A, B, D, E\}$, $|C_{\text{match}}| = 4$.

Óptimo de referencia (por inspección): $C^* = \{A, C, E\}$ cubre las 6 aristas, por lo tanto $|C^*| = 3$.

Algoritmo	Cobertura	Tamaño	Respecto al óptimo (3)
Greedy por grado	$\{A, B, E\}$	3	Óptimo en esta instancia
Matching maximal (2-aprox)	$\{A, B, D, E\}$	4	1.33× el óptimo
Exacto (referencia)	$\{A, C, E\}$	3	—

Este contraste – heurística sin garantía formal pero buen desempeño práctico, frente a aproximación con garantía matemática pero resultado subóptimo en casos concretos – es el eje de la comparación experimental desarrollada en este informe.

9 Diseño algorítmico

9.1 Greedy por grado (greedy_degree_vertex_cover)

Algorithm 1 greedy_degree_vertex_cover

```

1:  $E' \leftarrow E$  ▷ aristas pendientes
2:  $C \leftarrow \emptyset$ 
3: while  $E' \neq \emptyset$  do
4:   calcular  $\deg_{E'}(v)$  para cada  $v$  incidente en  $E'$ 
5:    $v^* \leftarrow \arg \max_v \deg_{E'}(v)$ 
6:    $C \leftarrow C \cup \{v^*\}$ 
7:    $E' \leftarrow E' \setminus \{(u, w) \in E' : u = v^* \vee w = v^*\}$ 
8: end while
9: return  $C$ 

```

Invariante de ciclo: al inicio de cada iteración, E' es exactamente el conjunto de aristas de E no cubiertas por C . Esto se cumple trivialmente al inicio ($C = \emptyset$, $E' = E$) y se preserva en cada paso, porque al agregar v^* a C se eliminan de E' exactamente las aristas que v^* cubre y ninguna otra.

Corrección: el ciclo termina cuando $E' = \emptyset$, es decir, cuando toda arista de E tiene al menos un extremo en C (por el invariante). Por lo tanto C devuelto es siempre una cobertura de vértices válida.

Complejidad: en la implementación actual (`src/solver.py`), cada iteración recorre las aristas restantes para recalcular grados: $O(E)$ por iteración, hasta $O(V)$ iteraciones en el peor caso $\Rightarrow O(V \cdot E)$. Puede optimizarse a $O((V + E) \log V)$ manteniendo una cola de prioridad; queda identificado como mejora pendiente para la Entrega 2 (Sección 15).

Garantía de aproximación: ninguna cota fija demostrable en el peor caso general (existen instancias donde el greedy se aleja más del óptimo que el matching maximal); en la práctica, sobre las instancias probadas en este periodo, el ratio empírico se mantuvo entre 1.0x y 1.2x (Sección 13).

9.2 Matching maximal / 2-aproximación (maximal_matching_vertex_cover)

Algorithm 2 maximal_matching_vertex_cover

```

1:  $E' \leftarrow E$ 
2:  $C \leftarrow \emptyset$ 
3: while  $E' \neq \emptyset$  do
4:   tomar  $(u, v) \in E'$  ▷ orden canónico, ver nota de reproducibilidad
5:    $C \leftarrow C \cup \{u, v\}$ 
6:    $E' \leftarrow E' \setminus \{(a, b) \in E' : a \in \{u, v\} \vee b \in \{u, v\}\}$ 
7: end while
8: return  $C$ 

```

Invariante: las aristas elegidas en pasos sucesivos del ciclo $(u_1, v_1), (u_2, v_2), \dots$ nunca comparten un extremo entre sí, porque cada elección elimina de E' todas las aristas incidentes a los dos vértices recién agregados antes de la siguiente elección. Es decir, el conjunto de aristas elegidas forma siempre un *matching* (emparejamiento) del grafo.

Corrección: igual que en el greedy, el ciclo termina solo cuando $E' = \emptyset$, así que C cubre toda arista de E .

Complejidad: $O(V + E)$: cada arista se visita y se elimina a lo sumo una vez del conjunto de pendientes.

Demostración de la garantía de aproximación. Sea k el número de aristas elegidas por el algoritmo (el tamaño del matching). Como esas k aristas son disjuntas dos a dos (invariante anterior), ningún vértice puede cubrir más de una de ellas. Por lo tanto, *toda* cobertura de vértices válida – incluida la óptima C^* – debe contener al menos un extremo distinto de cada una de esas k aristas, de donde $|C^*| \geq k$. El algoritmo agrega los dos extremos de cada una de las k aristas, por lo que $|C_{\text{match}}| = 2k$. Combinando ambas desigualdades:

$$|C_{\text{match}}| = 2k \leq 2 \cdot |C^*|.$$

9.3 Exacto (fuerza bruta de referencia) (exact_vertex_cover_bruteforce)

Algorithm 3 exact_vertex_cover_bruteforce

```

1: for  $k = 1$  hasta  $|V|$  do
2:   for cada subconjunto  $S \subseteq V$  con  $|S| = k$  do
3:     if  $S$  cubre todas las aristas de  $E$  then
4:       return  $S$ 
5:     end if
6:   end for
7: end for
```

Corrección y optimalidad: el algoritmo prueba exhaustivamente todos los subconjuntos de tamaño $k = 1, 2, \dots$ en orden creciente, y devuelve el primero que sea una cobertura válida. Como se prueban *todos* los subconjuntos de cada tamaño antes de pasar al siguiente, el primer subconjunto válido encontrado tiene necesariamente el tamaño mínimo posible, es decir, es una cobertura óptima.

Complejidad: $O\left(\binom{V}{k^*} \cdot E\right)$ en el peor caso, acotado superiormente por $O(2^V \cdot E)$. Por esta razón, en la implementación, `exact_vertex_cover_bruteforce` rechaza instancias de más de 20 vértices (parámetro configurable) lanzando `ValueError` en vez de ejecutarse indefinidamente. Los datos de la Sección 13 confirman el crecimiento exponencial esperado incluso dentro de ese límite.

Nota de reproducibilidad. Durante la implementación se detectó que iterar directamente sobre un `set()` de aristas en Python produce un orden distinto en cada ejecución del proceso (por la aleatorización del hash de los *strings*, `PYTHONHASHSEED`), lo que rompía la reproducibilidad exigida por el proyecto (instrucción adicional 7). Se corrigió introduciendo el método `_sorted_edges()` de `VertexCoverSolver`, que fija un orden canónico y determinístico para los desempates del greedy y la elección "arbitraria" del matching maximal. Este hallazgo se analiza con más detalle en la Sección 14.

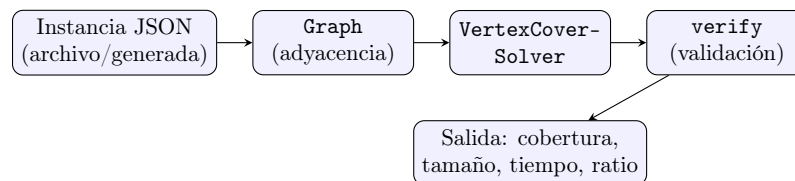
10 Diseño e implementación del sistema

10.1 Arquitectura y módulos

El sistema se organiza en módulos con responsabilidades separadas, de modo que la lógica algorítmica no dependa de cómo se presenta o se invoca:

Módulo	Archivo	Responsabilidad
Grafo	<code>src/graph.py</code>	Estructura de datos, carga/guardado de instancias, generadores sintéticos
Solver	<code>src/solver.py</code>	Los tres algoritmos (greedy, matching, exacto)
Verificación	<code>src/verify.py</code>	Validación de coberturas y cálculo de métricas
Pruebas	<code>tests/</code>	Casos básicos, límite y adversarios (pytest)
Experimentos	<code>experiments/compare.py</code>	Comparación empírica y exportación a CSV
Prototipo/demo	<code>demo.py</code>	Interfaz de línea de comandos para la demostración

10.2 Flujo de información



10.3 Estructura de datos

Se usa una **lista de adyacencia** (`dict[vértice, set[vértice]]`) en lugar de una matriz de adyacencia:

- Las redes reales son dispersas ($|E| \ll |V|^2$), por lo que una matriz desperdiciaría memoria: $O(V^2)$ vs. $O(V + E)$ de la lista de adyacencia.
- Obtener el grado de un vértice es $O(1)$ (tamaño del conjunto de vecinos), operación central en el algoritmo greedy.
- Insertar/eliminar una arista es $O(1)$ amortizado gracias al uso de `set`.

10.4 Formato de archivo de instancia

Las instancias se guardan en JSON, elegido por ser legible por humanos, nativo en Python y fácil de versionar en Git (diffs claros por línea):

```

{
  "name": "example_small",
  "description": "...",
  "vertices": ["A", "B", "C", "D", "E", "F"],
  "edges": [["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["D", "E"], ["E", "F"]]
}
  
```

10.5 Dependencias y decisiones técnicas

El proyecto depende de `pytest` (pruebas), `networkx`, `matplotlib` y `pandas` (candidatas para la Entrega 2, declaradas en `requirements.txt` pero aún no integradas al núcleo) y `pulp` (pendiente de completar su integración). El núcleo algorítmico (`src/`) no depende de ninguna librería externa, solo de la biblioteca estándar de Python, por decisión explícita: mantener el núcleo auditable y sin puntos de fallo externos, dejando las dependencias externas para capas periféricas (visualización, experimentación a mayor escala).

11 Estrategia de pruebas

El plan de pruebas está implementado en `tests/` (27 casos, `pytest`) y se organiza en las tres categorías pedidas por la guía:

Categoría	Casos	Archivo
Básicos	Construcción del grafo; instancia del Avance 1 (resultado exacto esperado); triángulo K_3 (óptimo conocido = 2).	<code>test_graph.py</code> , <code>TestCasos-</code> <code>Basicos</code>
Límite	Grafo sin aristas; una sola arista (evidencia directa de la cota $2x$); vértices aislados; grafo completo K_4 (óptimo = $n - 1$); componentes desconectadas; fuerza bruta rechazando instancias > 20 vértices.	<code>TestCasosLimite</code>
Adversarios	Grafo estrella (greedy óptimo, matching subóptimo pero válido); verificación parametrizada en 10 grafos aleatorios de que el matching maximal nunca supera $2\times$ el óptimo; verificación de que ninguna arista queda sin cubrir.	<code>TestCasos-</code> <code>Adversarios</code>

11.1 Criterio de verificación de salidas

Ninguna prueba confía únicamente en el *tamaño* de la cobertura devuelta. Toda salida se valida con `is_vertex_cover(graph, cover)` (`src/verify.py`), que recorre todas las aristas del grafo y confirma que cada una tenga al menos un extremo en la cobertura. Esto es intencional: un algoritmo con un error de implementación podría devolver un conjunto del tamaño "esperado" sin que efectivamente cubra el grafo, y una prueba que solo comparara tamaños no lo detectaría – precisamente el tipo de riesgo que produjo el defecto de reproducibilidad descrito en la Sección 14.

11.2 Pruebas de integración y validación cruzada

Además de las pruebas unitarias por algoritmo, `TestCasosAdversarios` incluye una verificación de integración: sobre 10 instancias aleatorias distintas, se ejecutan los tres algoritmos y se compara el resultado del matching maximal contra el exacto, confirmando que la cota teórica $|C_{\text{match}}| \leq 2|C^*|$ se cumple empíricamente en cada caso, no solo en la instancia manual del Avance 1.

11.3 Resultado de la ejecución

```
$ pytest tests/ -v
...
===== 27 passed in 0.04s =====
```

12 Metodología experimental

12.1 Preguntas de evaluación

1. ¿Qué tan cerca del óptimo queda cada algoritmo aproximado, en la práctica, frente a su cota teórica?
2. ¿Cómo escala el tiempo de ejecución de cada algoritmo con el tamaño de la instancia?
3. ¿A partir de qué tamaño de instancia el algoritmo exacto deja de ser una referencia práctica?

12.2 Hipótesis de trabajo

Se espera que el greedy por grado obtenga, en promedio, un ratio de aproximación empírico más cercano a 1.0 que el matching maximal (pese a no tener garantía teórica), y que el tiempo del algoritmo exacto crezca de forma exponencial con $|V|$, volviéndose impráctico mucho antes de alcanzar el límite de 20 vértices configurado en el código.

12.3 Instancias, variables y métricas

- **Instancias:** grafos aleatorios $G(n, p)$ (modelo Erdős–Rényi), generados con la función `random_graph` de `Graph` (archivo `src/graph.py`), con semilla fija por tamaño para reproducibilidad.
- **Variable independiente:** número de vértices $n \in \{6, 8, 10, 12, 14, 16, 18\}$, con densidad $p = 0.3$ fija en esta primera ronda.
- **Métricas:** tamaño de la cobertura obtenida, ratio de aproximación empírico $|C_{\text{algoritmo}}|/|C^*|$ (calculado por `verify.approximation_ratio`), y tiempo de ejecución en milisegundos (`time.perf_counter()`).

12.4 Entorno y procedimiento reproducible

Los experimentos se ejecutan con `python experiments/compare.py` desde la raíz del repositorio, sin argumentos ni configuración adicional. El script fija las semillas aleatorias internamente (`SEED_BASE + n`), por lo que una misma ejecución en cualquier máquina con las dependencias de `requirements.txt` instaladas debe producir exactamente los mismos resultados (condición que no se cumplía antes de corregir el defecto de reproducibilidad de la Sección 14). El resultado se guarda en `experiments/results/comparison.csv`.

13 Resultados

13.1 Validación contra la instancia manual

Se ejecuta `demo.py` sobre la instancia `data/instances/example_small.json` (la mini-red de 6 routers de la Sección 8.7):

```
$ python demo.py --instance data/instances/example_small.json --algorithm all

Cargado: instancia 'data/instances/example_small.json'
|V| = 6 |E| = 6

[Greedy por grado]
Cobertura: ['A', 'B', 'E'] Tamano: 3 Valida: SI Tiempo: 0.034 ms

[Matching maximal (2-aprox)]
Cobertura: ['A', 'B', 'D', 'E'] Tamano: 4 Valida: SI Tiempo: 0.018 ms

[Exacto (fuerza bruta)]
Cobertura: ['A', 'B', 'E'] Tamano: 3 Valida: SI Tiempo: 0.031 ms

Ratios de aproximacion frente al optimo:
Greedy por grado: 1.000x
Matching maximal (2-aprox): 1.333x
```

El resultado coincide exactamente con la traza manual de la Sección 8.7, lo que valida que la implementación respeta el diseño acordado.

13.2 Experimento comparativo

Primera corrida del script de experimentos (`experiments/compare.py`): grafos aleatorios $G(n, 0.3)$, $n \in \{6, 8, 10, 12, 14, 16, 18\}$, semilla fija por tamaño.

n	$ E $	óptimo	greedy (ratio)	matching (ratio)	t_{exacto} (ms)
6	3	2	2 (1.00x)	4 (2.00x)	0.02
8	10	4	4 (1.00x)	6 (1.50x)	0.06
10	13	5	6 (1.20x)	10 (2.00x)	0.31
12	19	5	5 (1.00x)	10 (2.00x)	0.97
14	35	9	9 (1.00x)	12 (1.33x)	9.08
16	33	9	9 (1.00x)	14 (1.56x)	29.61
18	52	12	12 (1.00x)	16 (1.33x)	173.90

Observaciones:

- El tiempo del algoritmo exacto crece de forma visiblemente exponencial incluso en este rango pequeño (factor $\approx 6\times$ entre $n=16$ y $n=18$).
- El greedy iguala al óptimo en 6 de las 7 instancias; solo se desvía (ratio 1.2x) en $n=10$.
- El matching maximal se mantiene siempre dentro de su cota teórica (máximo observado: 2.0x, exactamente el límite superior), pero nunca superó al greedy en estas instancias.

14 Discusión crítica

14.1 Interpretación de los resultados frente a la teoría

Los resultados son consistentes con lo esperado teóricamente: el matching maximal nunca excedió su cota de $2 \times$ el óptimo en ninguna de las instancias probadas (ni en las 7 del experimento comparativo, ni en las 10 adicionales verificadas automáticamente en `TestCasosAdversarios`), lo cual es evidencia empírica – no una demostración, pero sí un indicio fuerte – de que la implementación respeta la garantía demostrada en la Sección 9. Que el greedy supere casi siempre al matching maximal en estas instancias no contradice la teoría: el greedy simplemente carece de una cota garantizada, lo que significa que *podría* comportarse peor en otras instancias (por ejemplo, instancias construidas específicamente para explotar su debilidad), no que sea estrictamente mejor en general. Confirmarlo o refutarlo con instancias adversarias diseñadas a propósito para el greedy queda como trabajo pendiente para la Entrega 2.

14.2 El hallazgo del periodo: no determinismo en la implementación inicial

El hallazgo técnico más relevante de este periodo no aparece en las tablas de resultados, sino en el proceso: al escribir las pruebas del Avance 2, se detectó que dos ejecuciones del mismo algoritmo sobre la misma instancia podían devolver coberturas *válidas pero de tamaño distinto*. La causa raíz, investigada por el grupo, es que Python aleatoriza el hash de los objetos `str` entre procesos distintos (`PYTHONHASHSEED`), por lo que el orden de iteración de un `set()` no está garantizado entre ejecuciones. Ambos algoritmos iteraban directamente sobre conjuntos de aristas, heredando ese no-determinismo.

Esto es importante por dos razones, más allá de ser un simple error de programación: primero, viola directamente la instrucción adicional 7 de la guía del proyecto ("el repositorio deberá incluir [...] semillas aleatorias, cuando corresponda" – la reproducibilidad es un requisito explícito, no una buena práctica opcional); segundo, sin este arreglo, ningún resultado experimental de la Sección 13 sería confiable, porque no se podría distinguir una variación real de desempeño de un simple artefacto de la aleatorización del hash. Se corrigió introduciendo un orden canónico fijo (`_sorted_edges`), verificado manualmente variando `PYTHONHASHSEED` entre tres valores distintos y confirmando resultados idénticos en los tres casos.

14.3 Limitaciones y amenazas a la validez

- **Una sola densidad de grafo.** El experimento comparativo usó únicamente $p = 0.3$; no se sabe todavía cómo cambian los ratios con grafos más dispersos o más densos, ni con topologías distintas a Erdős–Rényi (por ejemplo, la topología tipo backbone con hubs de alto grado ya disponible en `data/instances/network_backbone_18.json`, generada pero aún no incluida en el experimento sistemático).
- **Rango de n limitado por la fuerza bruta.** El óptimo de referencia solo se pudo calcular hasta $n = 18$ en tiempos razonables; no hay todavía evidencia empírica del ratio de aproximación en instancias mayores, donde seguiría siendo necesario el solver ILP (pendiente).
- **Una sola corrida por configuración.** Cada punto de la tabla de resultados proviene de una única instancia por tamaño (con semilla fija), no de un promedio sobre varias instancias del mismo tamaño; el ratio observado para un n dado podría no ser representativo de todas las instancias de ese tamaño.

14.4 Cuándo conviene cada alternativa

Con la evidencia disponible hasta ahora: el **exacto** solo es viable para instancias muy pequeñas (en este entorno, claramente impráctico más allá de $n \approx 18-20$) y debe usarse exclusivamente como referencia de validación, nunca en producción. El **greedy** es la opción recomendable cuando se prioriza la calidad de la solución sobre la garantía formal, dado su buen desempeño empírico observado. El **matching maximal** es preferible cuando se necesita una *garantía demostrable* sobre el peor caso – por ejemplo, en un contexto donde no poder razonar sobre una cota superior sea inaceptable – incluso a costa de una solución probablemente más grande en la práctica.

15 Conclusiones y recomendaciones

15.1 Respuesta a los objetivos

Los objetivos específicos planteados en la Introducción se cumplieron para el alcance de esta entrega: el problema está formulado formalmente, el diseño del sistema está implementado y verificado, se implementaron tres algoritmos (dos más que el mínimo exigido de dos comparables) con sus garantías demostradas donde corresponde, el plan de pruebas cubre los tres tipos de casos exigidos con 27 pruebas pasando, el prototipo es ejecutable y demostrable en vivo, y se ejecutó un primer experimento reproducible con observaciones documentadas.

15.2 Síntesis de hallazgos

El resultado más valioso del periodo no fue un número de la tabla de resultados, sino el proceso mismo de encontrar y corregir el defecto de reproducibilidad: evidencia directa de que el plan de pruebas cumple su propósito (detectar errores reales, no solo confirmar lo esperado) y de que el grupo puede diagnosticar y resolver un problema técnico no trivial sin supervisión externa.

15.3 Lecciones aprendidas

- Escribir las pruebas adversarias *antes* de confiar en los resultados experimentales permitió detectar un error que un experimento aislado, ejecutado una sola vez, nunca habría revelado.
- La reproducibilidad no es automática por el solo hecho de fijar una semilla aleatoria explícita: estructuras de datos aparentemente inocuas como `set()` pueden introducir no-determinismo oculto si no se tiene cuidado con el orden de iteración.
- Priorizar el núcleo algorítmico sobre el acabado de las herramientas periféricas (como se hizo con la limitación de $\text{\LaTeX}$) es consistente con el criterio explícito de la guía del proyecto y evitó perder tiempo en un problema no esencial.

15.4 Trabajo futuro (Entrega 2)

1. Completar e integrar `exact_vertex_cover_ilp` (PuLP) como referencia escalable más allá del límite de la fuerza bruta.
2. Optimizar `greedy_degree_vertex_cover` a $O((V + E) \log V)$ con una cola de prioridad, verificando con una prueba que ambas versiones produzcan el mismo resultado.
3. Ampliar la metodología experimental: variar la densidad p además de n , e incluir instancias tipo backbone (Barabási–Albert), no solo Erdős–Rényi; repetir cada configuración sobre varias instancias, no solo una.
4. Incorporar visualización de grafos pequeños (candidato: NetworkX + matplotlib).
5. Instrumentar uso de memoria como métrica adicional.
6. Diseñar instancias adversarias dirigidas específicamente a explotar la falta de garantía formal del greedy, para contrastar con el buen desempeño observado hasta ahora.

16 Referencias

References

- [1] Karp, R. M. (1972). *Reducibility Among Combinatorial Problems*. En R. E. Miller y J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press.
- [2] Vazirani, V. V. (2003). *Approximation Algorithms*. Springer-Verlag.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3.^a ed.). MIT Press.
- [4] pytest documentation. <https://docs.pytest.org/>. Consultado en 2026.
- [5] PuLP documentation – COIN-OR Foundation. <https://coin-or.github.io/pulp/>. Consultado en 2026.
- [6] NetworkX documentation. <https://networkx.org/documentation/stable/>. Consultado en 2026.

A Anexos

A.1 Manual de instalación

```
git clone <url-del-repositorio>
cd vertex-cover-ids
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

A.2 Manual de uso

```
# Correr las pruebas
pytest tests/ -v

# Prototipo sobre una instancia conocida
python demo.py --instance data/instances/example_small.json --algorithm all

# Prototipo sobre una instancia nueva generada al vuelo
python demo.py --random 15 0.25 --seed 42 --algorithm all

# Experimento comparativo
python experiments/compare.py
```

A.3 Bitácora de aprendizaje (resumen)

El registro completo, con fecha y detalle por sesión, está en `docs/bitacora.md` del repositorio. Resumen de las tres sesiones realizadas en este periodo:

- **Sesión 1:** formulación del problema, formalización de entradas/salidas/ restricciones, resolución manual de la instancia de ejemplo con ambos algoritmos acordados.
- **Sesión 2:** implementación del núcleo (`Graph`, `VertexCoverSolver`, `verify`), plan de pruebas de 27 casos; hallazgo y corrección del defecto de reproducibilidad descrito en la Sección 14.
- **Sesión 3:** prototipo ejecutable (`demo.py`), primer experimento comparativo reproducible, y documentación de las tres entregas.

Nota: esta matriz debe completarse con nombres reales y evidencia verificable (commits de Git, autoría de pruebas o secciones del documento) antes de la presentación, ya que el atributo AG_C06 se mide de forma individual aunque el trabajo sea grupal.

A.4 Evidencia adicional

El repositorio Git del proyecto contiene el historial completo de commits, que respalda la evolución incremental descrita en este informe (estructura inicial → estructura de grafo → algoritmos → corrección del defecto de reproducibilidad → pruebas → prototipo → experimento → documentación de cada avance). El resultado crudo del experimento de la Sección 13 está disponible en `experiments/results/comparison.csv`.